After Training, Testing and Collecting New Signs

Publish verified static and continuous sign models to the live website

SAHAYAK | COPY-PASTE FIELD GUIDE

Publish your verified static and continuous sign models to the live website.

> **PURPOSE: use this guide after new signs have been collected, trained, tested and promoted locally. It takes you from the active local model files to GitHub, Render and final live-camera testing.**

Your verified folder setup

- Training project: C:\Users\user\Videos\SAHAYAK
- GitHub clone: C:\Users\user\Documents\GitHub\SAHAYAK-ISL
- Static model: model\sahayak_twohand.joblib - 126 input features.
- Continuous model: model\sahayak_sequence.joblib - 1,320 input features.

Safe publishing rules

- Publish only promoted active .joblib model files, never candidate models.
- Do not publish raw CSV files, raw sequence samples, or the training database.
- Do not change recognition code, API routes, camera logic, Render settings or Vercel settings for a model-only update.

> **The static and continuous classifiers are separate. Never copy one model over the other.**

Run in VS Code PowerShell. Use the same terminal through this workflow. If you moved either folder, edit its path before running.

CHECK PROJECT AND UPDATE MAIN

```
$ErrorActionPreference = "Stop"
$TrainingRoot = "C:\Users\user\Videos\SAHAYAK"
$RepoRoot = "C:\Users\user\Documents\GitHub\SAHAYAK-ISL"
$TrainingPython = Join-Path $TrainingRoot ".venv\Scripts\python.exe"
if (-not (Test-Path $TrainingPython)) { throw "Training venv not found." }
if (-not (Test-Path (Join-Path $RepoRoot ".git"))) { throw "Git clone not found." }
Set-Location $RepoRoot
$Remote = (git remote get-url origin).Trim()
if ($Remote -notmatch 'github\.com[:/]Mrjohanjyotikuli/SAHAYAK-ISL(\.git)?$') {
    throw "This is not the expected SAHAYAK-ISL clone."
}
if (@(git status --porcelain).Count -gt 0) {
    throw "Repository is not clean. Review changes before continuing."
}
git pull --ff-only origin main
if ($LASTEXITCODE -ne 0) { throw "Could not update main." }
Write-Host "Project and GitHub remote verified."
```

If the clone is not clean, stop and review the existing changes instead of overwriting them.

2. Confirm that local collection, testing and promotion are complete

- Check that you have finished testing the candidate and promoted the approved model.
- Use the active filenames below, not files ending in _candidate.joblib.

This check loads the active models with your training virtual environment and prints every model label. Stop if a feature count is wrong or any expected class is missing.

CHECK FEATURES AND LABELS

```
$TrainingRoot = "C:\Users\user\Videos\SAHAYAK"
$TrainingPython = Join-Path $TrainingRoot ".venv\Scripts\python.exe"
@'
import joblib
from pathlib import Path
root = Path(r"C:\Users\user\Videos\SAHAYAK")
checks = [
    ("STATIC", root / "model" / "sahayak_twohand.joblib", 126),
    ("CONTINUOUS", root / "model" / "sahayak_sequence.joblib", 1320),
]
for name, path, expected in checks:
    if not path.is_file():
        raise SystemExit(f"Missing active model: {path}")
    model = joblib.load(path)
    features = getattr(model, "n_features_in_", None)
    labels = sorted(str(x) for x in model.classes_)
    print(f"\n{name}: {path.name}")
    print("Features:", features)
    print("Label count:", len(labels))
    print("Labels:", labels)
    if features != expected:
        raise SystemExit(f"Wrong feature count for {name}")
'@ | & $TrainingPython -
if ($LASTEXITCODE -ne 0) { throw "Local model verification failed." }
```

Expected dimensions: 126 for static signs and 1,320 for continuous signs. Check the printed labels before copying anything.

This block backs up the old deployment copies outside the GitHub clone, copies the active models and verifies SHA-256 hashes.

BACK UP, COPY AND VERIFY

```
$ErrorActionPreference = "Stop"
$TrainingRoot = "C:\Users\user\Videos\SAHAYAK"
$RepoRoot = "C:\Users\user\Documents\GitHub\SAHAYAK-ISL"
$SourceDir = Join-Path $TrainingRoot "model"
$TargetDir = Join-Path $RepoRoot "SAHAYAK-Vercel-Deployment\backend\model"
$Names = @("sahayak_twohand.joblib", "sahayak_sequence.joblib")
foreach ($Name in $Names) {
    if (-not (Test-Path (Join-Path $SourceDir $Name))) { throw "Missing $Name" }
}
if (-not (Test-Path $TargetDir)) { throw "Deployment model folder missing." }
$Stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$BackupDir = Join-Path $TrainingRoot "backups\deployment_models_$Stamp"
New-Item -ItemType Directory -Force $BackupDir | Out-Null
foreach ($Name in $Names) {
    $Target = Join-Path $TargetDir $Name
    if (Test-Path $Target) { Copy-Item $Target $BackupDir -Force }
}
Write-Host "Backup saved to $BackupDir"
foreach ($Name in $Names) {
    $Source = Join-Path $SourceDir $Name
    $Target = Join-Path $TargetDir $Name
    Copy-Item $Source $Target -Force
    $A = (Get-FileHash $Source -Algorithm SHA256).Hash
    $B = (Get-FileHash $Target -Algorithm SHA256).Hash
    if ($A -ne $B) { throw "Hash mismatch: $Name" }
    Write-Host "Verified copy: $Name" -ForegroundColor Green
}
Set-Location $RepoRoot
git status --short
```

Only these active models should be copied: sahayak_twohand.joblib and sahayak_sequence.joblib. Never copy candidate model files.

This guarded block permits one or both models to have changed, but refuses to continue if unrelated files are modified or already staged.

SAFE MODEL-ONLY COMMIT AND PUSH

```
$ErrorActionPreference = "Stop"
$RepoRoot = "C:\Users\user\Documents\GitHub\SAHAYAK-ISL"
Set-Location $RepoRoot
$Allowed = @(
    "SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib",
    "SAHAYAK-Vercel-Deployment/backend/model/sahayak_sequence.joblib"
)
if ((git branch --show-current).Trim() -ne "main") { throw "Not on main." }
if (@(git diff --cached --name-only).Count -gt 0) {
    throw "Something is already staged. Review it first."
}
$Status = @(git status --porcelain)
$Unexpected = @($Status | Where-Object { $_.Substring(3).Trim('"') -notin $Allowed })
if ($Unexpected.Count -gt 0) {
    $Unexpected | ForEach-Object { Write-Host $_ -ForegroundColor Red }
    throw "Unrelated changes found. Stop and review."
}
if ($Status.Count -eq 0) { throw "No model changes detected." }
git add -- $Allowed
if ($LASTEXITCODE -ne 0) { throw "git add failed." }
$Staged = @(git diff --cached --name-only)
$Bad = @($Staged | Where-Object { $_ -notin $Allowed })
if ($Staged.Count -eq 0 -or $Bad.Count -gt 0) { throw "Unexpected staged files." }
$Staged
git diff --cached --stat
git commit -m "Update static and continuous sign models"
if ($LASTEXITCODE -ne 0) { throw "Commit failed." }
git push origin main
if ($LASTEXITCODE -ne 0) { throw "Push failed." }
git log -1 --oneline
Write-Host "Models pushed to GitHub." -ForegroundColor Green
```

If Git says Author identity unknown, configure user.name and user.email for this repository only.
Use the email configured in GitHub Settings > Emails; do not put a password or token in a command.

Your Render backend deploys from main when a new commit is pushed. Open the Render dashboard and wait for the latest deployment to show Live before running the status check.

Render dashboard: https://dashboard.render.com/web/srv-dbaof849v7es739am2j0

CHECK LIVE MODEL STATUS

```
$ErrorActionPreference = "Stop"
$Status = Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status"
Write-Host "Static model ready:" $Status.model_ready
Write-Host "Static class count:" $Status.single_sign_label_count
Write-Host "Static labels:"
$Status.single_sign_labels
Write-Host "Continuous model ready:" $Status.sequence_model_ready
Write-Host "Continuous class count:" $Status.sequence_sign_label_count
Write-Host "Continuous labels:"
$Status.sequence_sign_labels
```

Success checks

- model_ready is True.
- sequence_model_ready is True.
- Both label lists contain every expected retained and newly trained sign.

A previously verified build reported 8 static labels and 6 continuous labels. Future updates may change the totals, so always inspect the label names rather than relying on counts alone.

Live website: https://sahayak-eight-lac.vercel.app/

Camera smoke-test checklist

- Hard-refresh with Ctrl + Shift + R, allow camera access and choose the correct camera.
- Test every new static sign with the intended hand pose.
- Test every new continuous sign using the full movement, not a still frame.
- Test several old signs to confirm retained classes still work.
- Check the recognized label, landmarks/annotation and prediction stability.

If something fails

- If Render is still deploying, wait until Live and rerun the status check.
- If a label is missing, compare local active labels and confirm you promoted the intended model before copying.
- If Git push is rejected, do not force-push. Review git status --short and the full error.
- If backend readiness is False, inspect the latest Render deployment logs before testing the camera.

Repeatable workflow

- Collect varied samples; train a candidate; test with fresh examples; inspect all labels; promote only after passing checks.
- Copy active model file(s); back up old deployment copies; stage only model files; push main; wait for Render Live; check API status and test the live camera.

> **This guide covers publish-and-verify steps after local collection, training, testing and promotion. It does not require a frontend redeploy when only backend model files changed.**